Aulão Algoritmos e Estrutura de Dados II

1. **Análise do Algoritmo de Seleção**
   Analise o seguinte algoritmo de ordenação por seleção e determine sua complexidade assintótica em termos do número de comparações:

```java
void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n-1; i++) {
        int minIndex = i;
        for (int j = i+1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
```

2. **Análise de Algoritmo com Loop Duplo**
   Analise o seguinte algoritmo e determine sua complexidade assintótica:

```java
void complexAlgorithm(int n) {
    int count = 0;
    for (int i = 1; i <= n; i *= 2) {
        for (int j = 1; j <= i; j++) {
            count++;
        }
    }
}
```

3.

```
int x = 0;

for(int i = 15; i< n; i ++){
   for(int j = 0; j < 3; j++) x += a * 2;
    for(int k = i; k<n; k ++){
       x += b * 5;
       x += x * 10;
   }
    x+ = c*4;


}

```

a) A função de complexidade usando o somatório para o número de multiplicações.
b) A fórmula fechada da função de complexidade e a ordem de complexidade usando notação Θ.

4.

```
class Celula {
    public char elemento;
    public Celula prox;

    public Celula() {
        this('\0');
    }

    public Celula(char elemento) {
        this.elemento = elemento;
        this.prox = null;
    }
}

class ListaFlexivel {
    private Celula primeiro;
    private Celula ultimo;

    public ListaFlexivel() {
        primeiro = new Celula(); // Célula cabeça
        ultimo = primeiro;
    }

    public void inserirFim(char x) {}
    public void inserirInicio(char x) {}
    public void inserir(char x, int pos){}
    public char removerInicio() throws Exception {}
    public char removerFim() throws Exception {}
    public char remover(int pos) {}
    public void mostrar(){}

}
```

Implemente o método boolean isPalindromo(String s) na sua classe, utilizando **obrigatoriamente** a classe ListaFlexivel para verificar se a String s é um palíndromo. Se utilizar uma das funções da lista flexível a mesma deverá ser implementada.

5.
Ordenação em Estruturas Lineares - Insertion Sort: Considere a implementação de uma **Fila Circular** de inteiros em linguagem C, conforme as variáveis globais e o método swap fornecidos abaixo:

```c
#define MAXTAM 50
int array[MAXTAM + 1];
int primeiro, ultimo;
```

Implemente o método `void insercao()`, que deve ordenar os elementos atualmente presentes na fila utilizando o algoritmo de **Ordenação por Inserção (Insertion Sort)**.

Requisitos de Implementação:

- O algoritmo deve percorrer apenas os elementos válidos da fila (do `primeiro` ao `ultimo`).
- **Justifique** como você adaptou o Insertion Sort para funcionar em um array circular.

6. Considere a implementação em Java abaixo. `topo` aponta para o último elemento inserido e não há célula cabeça.

```java
class Celula {
    public int elemento;
    public Celula prox;

    public Celula(int x) {
        this.elemento = x;
        this.prox = null;
    }
}

class Pilha {
    private Celula topo;

    public Pilha() {
        topo = null;
    }

    public void mostrar() {
        System.out.print("[ ");
        for (Celula i = topo; i != null; i = i.prox) {
            System.out.print(i.elemento + " ");
        }
        System.out.println("]");
    }
}
```

(a) Um aluno implementou o `inserir` assim:

```java
public void inserir(int x) {
    Celula tmp = new Celula(x);
    topo = tmp;
    tmp.prox = topo;
    tmp = null;
}
```

Partindo de uma pilha vazia, desenhe as células e os ponteiros após `p.inserir(8)`; `p.inserir(2)`; e explique o que acontece ao chamar `p.mostrar()`. Depois, corrija o método.

(b) Implemente `void inverter()`, que inverte a ordem dos elementos da pilha (ex.: `topo → 4 → 1 → 3` vira `topo → 3 → 1 → 4`). Não é permitido criar células (`new`) nem usar estruturas auxiliares, como outra pilha ou arrays.

Soluções:

1. **Solução do Exercício 1**
   O algoritmo de seleção possui dois loops aninhados. O loop externo executa $n-1$ vezes.
   O loop interno executa $n-i-1$ vezes na iteração i.
   O número total de comparações é dado por:

$$\sum_{i=0}^{n-2}(n-i-1) = \sum_{j=1}^{n-1} j = \frac{(n-1)n}{2} = \Theta(n^2)$$

   Portanto, a complexidade é $\Theta(n^2)$.

2. **Solução do Exercício 2**
   O loop externo executa $\lfloor \log_2 n \rfloor + 1$ vezes. O loop interno executa i vezes na iteração i.
   O número total de operações é dado por:

$$\sum_{k=0}^{\lfloor \log_2 n \rfloor} 2^k = 2^{\lfloor \log_2 n \rfloor + 1} - 1 = \Theta(n)$$

   Portanto, a complexidade é $\Theta(n)$.

3. Solução da Questão

a) Função de complexidade usando o somatório

Contagem de multiplicações (`*`) por iteração:

- **Linha 4:** `x += a * 2;` $\rightarrow 1$ multiplicação por ciclo do laço j (executado 3 vezes).
- **Linhas 6 e 7:** `x += b * 5;` e `x += x * 10;` $\rightarrow 2$ multiplicações por ciclo do laço k.
- **Linha 9:** `x += c * 4;` $\rightarrow 1$ multiplicação.

Expressão do somatório para $n > 15$:

$$f(n) = \sum_{i=15}^{n-1} \left(\left(\sum_{j=0}^{2} 1 \right) + \left(\sum_{k=i}^{n-1} 2 \right) + 1 \right)$$

Simplificando os termos internos:

$$f(n) = \sum_{i=15}^{n-1} (3 + 2(n-i) + 1) = \sum_{\mathbf{i=15}}^{\mathbf{n-1}} (\mathbf{4} + \mathbf{2}(\mathbf{n} - \mathbf{i}))$$

b) Fórmula fechada e Ordem de Complexidade

Resolução da Fórmula Fechada:

$$\begin{aligned} f(n) &= \sum_{i=15}^{n-1} 4 \; + \; 2 \sum_{i=15}^{n-1} (n-i) \\ &= 4(n-15) \; + \; 2 \sum_{u=1}^{n-15} u \quad (\text{aplicando } u = n-i) \\ &= 4(n-15) \; + \; 2 \cdot \frac{(n-15)(n-14)}{2} \\ &= 4n - 60 \; + \; (n^2 - 29n + 210) \\ &= \mathbf{n^2 - 25n + 150} \quad (\text{para } n \geq 15) \end{aligned}$$

Ordem de Complexidade:

$$\mathbf{O(n^2)}$$

4.

```java
public boolean isPalindromo(String s) throws Exception {
    ListaFlexivel lista = new ListaFlexivel();

    // 1. Insere todos os caracteres da string na lista flexível
    for (int i = 0; i < s.length(); i++) {
        lista.inserirFim(s.charAt(i));
    }

    boolean resp = true;

    // 2. Compara os caracteres das extremidades (início e fim)
    // Para ser palíndromo, o primeiro caracter deve ser igual ao último
    for (int i = 0; i < s.length() / 2 && resp; i++) {
        char inicio = lista.removerInicio();
        char fim = lista.removerFim();

        if (inicio != fim) {
            resp = false;
        }
    }

    return resp;
}
```

```java
public void inserirFim(char x) {
    ultimo.prox = new Celula(x);
    ultimo = ultimo.prox;
}

public char removerInicio() throws Exception {
    if (primeiro == ultimo) {
        throw new Exception("Erro ao remover!");
    }
    Celula tmp = primeiro.prox;
    char elemento = tmp.elemento;
    primeiro.prox = tmp.prox;
    if (tmp == ultimo) {
        ultimo = primeiro;
    }
    tmp.prox = null;
    tmp = null;
    return elemento;
}
```

```java
public char removerFim() throws Exception {
    if (primeiro == ultimo) {
        throw new Exception("Erro ao remover!");
    }
    Celula i;
    for (i = primeiro; i.prox != ultimo; i = i.prox);
    char elemento = ultimo.elemento;
    ultimo = i;
    ultimo.prox = null;
    return elemento;
}
```

5.

```c
// Converte a posição lógica i (0 = primeiro da fila) na posição real do array
int pos(int i) {
    return (primeiro + i) % (MAXTAM + 1);
}

void insercao() {
    int n = (ultimo - primeiro + MAXTAM + 1) % (MAXTAM + 1);  // nº de elementos

    for (int i = 1; i < n; i++) {
        int tmp = array[pos(i)];
        int j = i - 1;

        while (j >= 0 && array[pos(j)] > tmp) {
            array[pos(j + 1)] = array[pos(j)];  // desloca para a direita
            j--;
        }
        array[pos(j + 1)] = tmp;
    }
}
```

```c
// Laço anda de fato de primeiro até o ultimo
void insercao1() {
    int tam = MAXTAM + 1;
    if (primeiro == ultimo){ // Vazia

        int antesDoPrimeiro = (primeiro - 1 + tam) % tam;  // substitui o "j >= 0"

        for (int i = (primeiro + 1) % tam; i != ultimo; i = (i + 1) % tam) {
            int tmp = array[i];
            int j = (i - 1 + tam) % tam;                    // posição anterior a i

            while (j != antesDoPrimeiro && array[j] > tmp) {
                array[(j + 1) % tam] = array[j];
                j = (j - 1 + tam) % tam;                    // volta uma posição, circularmente
            }
            array[(j + 1) % tam] = tmp;
    }
    }
}
```

Análise de Complexidade

$$\mathbf{\Theta(N^2)}$$

- **Melhor Caso:** $\mathbf{\Theta(N)}$
  Ocorre quando a lista já está ordenada. O laço externo executa N vezes e o laço interno encerra na primeira verificação ($O(1)$).
- **Pior Caso:** $\mathbf{\Theta(N^2)}$
  Ocorre quando a lista está em ordem inversa. Para cada elemento i, o laço interno percorre todos os elementos já processados para encontrar a posição de inserção. O somatório de comparações resulta em uma Progressão Aritmética $1 + 2 + 3 + \cdots + (N - 1)$, gerando complexidade quadrática.

6. **(a)** O `topo` recebe `tmp` antes da ligação. Por isso, `tmp.prox = topo` faz a nova célula apontar para ela mesma. Depois de `inserir(8)`, a célula 8 aponta para si. Depois de `inserir(2)`, o `topo` passa para a célula 2, que também aponta para si. A célula 8 fica sem nenhuma referência: em Java o coletor de lixo a recolhe, e em C seria vazamento de memória.

(b)

```java
public void inverter() {
    Celula anterior = null;
    Celula atual = topo;
    while (atual != null) {
        Celula proximo = atual.prox; // guarda o resto da pilha
        atual.prox = anterior;       // inverte o ponteiro
        anterior = atual;
        atual = proximo;
    }
    topo = anterior;
}
```